UCAM

Tema 3

Programación con Jetpack Compose

Aplicaciones para Dispositivos Móviles

Daniel Valero Carreras

Grado en Ingeniería Informática



ÍNDICE

CONTENIDOS

1. Interfaz avanzada
2. Notificaciones
3. Sensores
4. Multimedia

1. Interfaz avanzada

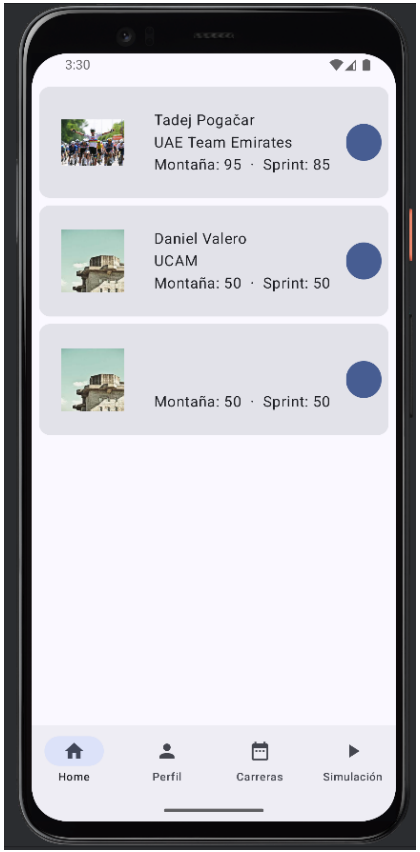
Imagen 1
Título asignado a la imagen



Nota. Adaptado de Título de la imagen, por Autor, Año, Web (con hipervínculo)

1. Interfaz avanzada
LazyColumn/LazyRow

- Un **LazyColumn/LazyRow** es un componente de Jetpack Compose para mostrar listas **verticales/horizontales y desplazables** de forma eficiente.
- **Pinta en pantalla los elementos que se ven** y va creando/reciclando el resto a medida que haces scroll
- Util para listas largas (chats, feeds, etc.), galerías de imágenes, etc.



1. Interfaz avanzada

LazyColumn

```
@Composable
fun ListaDeNombres(nombres: List<String>) {
    LazyColumn {
        items(nombres) { nombre ->
            Text(
                text = nombre,
                modifier = Modifier
                    .fillMaxWidth()
                    .padding(16.dp)
            )
        }
    }
}
```

1. Interfaz avanzada

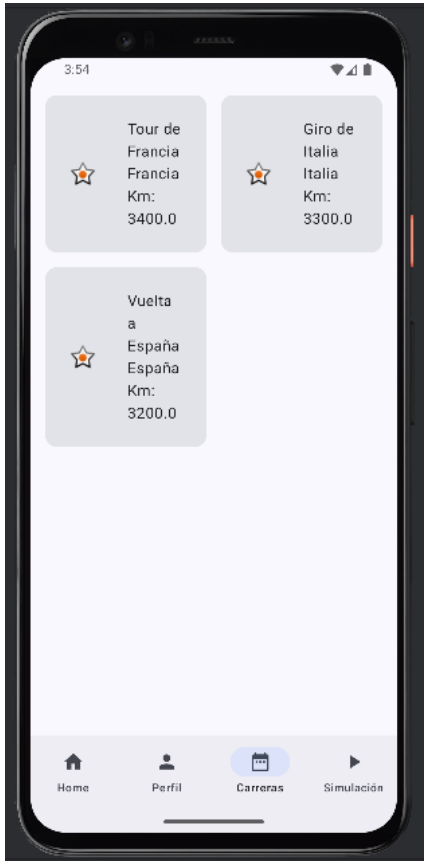
LazyRow

```
@Composable
fun ListaHorizontal() {
    val items = listOf("Android", "Kotlin", "Compose", "UI", "Material")

    LazyRow(
        contentPadding = PaddingValues(horizontal = 16.dp),
        horizontalArrangement = Arrangement.spacedBy(12.dp)
    ) {
        items(items) { item ->
            Card(
                shape = RoundedCornerShape(12.dp)
            ) {
                Text(
                    text = item,
                    modifier = Modifier
                        .padding(horizontal = 16.dp, vertical = 12.dp)
                )
            }
        }
    }
}
```

1. Interfaz avanzada
LazyVerticalGrid/LazyHorizontalGrid

- Muestra los elementos de forma vertical u horizontal en forma de rejilla
- El scroll es vertical u horizontal



1. Interfaz avanzada

LazyVerticalGrid

```
LazyVerticalGrid(
columns = GridCells.Fixed(2)  // 2 columnas
) {
   items(fotos) { foto ->
      Image(
         painter = painterResource(foto),
         contentDescription = null,
         modifier = Modifier
            .aspectRatio(1f)
            .padding(4.dp)
      )
   }
}
```

1. Interfaz avanzada

LazyHorizontalGrid

```
LazyHorizontalGrid(
rows = GridCells.Fixed(2)   // 2 filas
) {
    items(itemsLista) { item ->
        // contenido de cada celda
    }
}
```

1. Interfaz avanzada
Interfaz adaptativa

- Antes en XML se definían dos tipos de visualización en base a la orientación del dispositivo móvil
  - Portrait
  - Landscape
- Actualmente, se diseña una sola UI *adaptativa* que cambie según cuánto espacio tiene, no según si está girado o no.
- Para ello, utilizamos: *Window Size Classes* (Material 3)

1. Interfaz avanzada
Interfaz adaptativa

- Capturamos el tamaño de la ventana: calculateWindowSizeClass
- Tres tamaños:
  - Compact → móvil típico (vertical o horizontal pero estrecho)
  - Medium → móviles grandes / tablets pequeñas
  - Expanded → tablets / pantallas grandes

```
class MainActivity : ComponentActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        enableEdgeToEdge()
        setContent {
            DiseñoAdapTheme {
                val windowSizeClass = calculateWindowSizeClass(this)
                MyApp(windowSizeClass)
            }
        }
    }
}
```

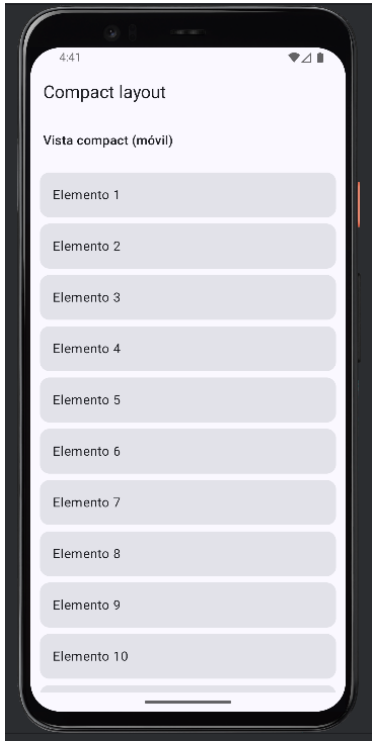
1. Interfaz avanzada

Interfaz adaptativa

```
@Composable
fun MyApp(windowSizeClass: WindowSizeClass) {
    when (windowSizeClass.widthSizeClass) {
        WindowWidthSizeClass.Compact -> {
            // Diseño para pantallas estrechas (móvil)
            CompactLayout()
        }
        WindowWidthSizeClass.Medium -> {
            // Quizá dos paneles
            MediumLayout()
        }
        WindowWidthSizeClass.Expanded -> {
            // Layout tipo tablet, 2–3 columnas
            ExpandedLayout()
        }
    }
}
```

1. Interfaz avanzada

Interfaz adaptativa





2. Notificaciones

Imagen 1
Título asignado a la imagen



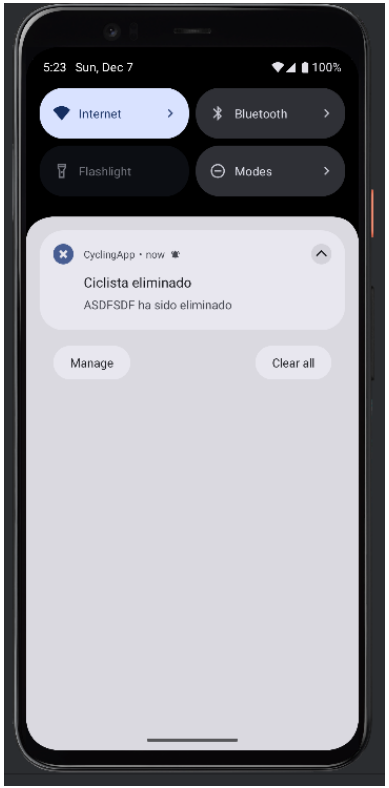
Esta foto de Autor desconocido está bajo licencia CC BY-SA

Nota. Adaptado de Título de la imagen, por Autor, Año, Web (con hipervínculo)

2. Notificaciones

¿Qué son?

- **Las notificaciones** son mensajes que tu app muestra **fuera de la interfaz** principal
- Se usan para avisar al usuario de algo importante cuando la app no está en primer plano (mensajes nuevos, recordatorios, descargas, etc.).
- Necesario:
  - Crear un canal de notificación (NotificationChannel)
  - Builder de notificación (NotificationCompat.Builder): define el icono, texto, etc.
  - NotificationManager (NotificationManagerCompat)
  - **PERMISOS** para notificar



2. Notificaciones

Canal de notificación

```
private const val CICLISTAS_CHANNEL_ID = "ciclistas_channel"
fun createCiclistasNotificationChannel(context: Context) {
    if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.O) {
        val name = "Ciclistas"
        val descriptionText = "Notificaciones relacionadas con ciclistas"
        val importance = NotificationManager.IMPORTANCE_DEFAULT
        val channel = NotificationChannel(CICLISTAS_CHANNEL_ID, name, importance).apply {
            description = descriptionText
        }
        val notificationManager: NotificationManager =
            context.getSystemService(Context.NOTIFICATION_SERVICE) as NotificationManager
        notificationManager.createNotificationChannel(channel)
    }
}
```

2. Notificaciones

Builder de notificación

```
fun showDeleteNotification(context: Context, ciclista: Ciclista) {
    // Aseguramos que el canal existe (Android 8+)
    createCiclistasNotificationChannel(context)

    val notification = NotificationCompat.Builder(context, CICLISTAS_CHANNEL_ID)
        .setSmallIcon(R.drawable.ic_delete) // usa un icono válido de tu app
        .setContentTitle("Ciclista eliminado")
        .setContentText("${ciclista.nombre} ha sido eliminado")
        .setPriority(NotificationCompat.PRIORITY_DEFAULT)
        .setAutoCancel(true)
        .build()

    with(NotificationManagerCompat.from(context)) {
        // ID cualquiera; podrías usar hashCode para diferenciar
        notify(ciclista.hashCode(), notification)
    }
}
```

2. Notificaciones

NotificationManager

```
fun showDeleteNotification(context: Context, ciclista: Ciclista) {
    // Aseguramos que el canal existe (Android 8+)
    createCiclistasNotificationChannel(context)

    val notification = NotificationCompat.Builder(context, CICLISTAS_CHANNEL_ID)
        .setSmallIcon(R.drawable.ic_delete) // usa un icono válido de tu app
        .setContentTitle("Ciclista eliminado")
        .setContentText("${ciclista.nombre} ha sido eliminado")
        .setPriority(NotificationCompat.PRIORITY_DEFAULT)
        .setAutoCancel(true)
        .build()

    with(NotificationManagerCompat.from(context)) {
        // ID cualquiera; podrías usar hashCode para diferenciar
        notify(ciclista.hashCode(), notification)
    }
}
```

2. Notificaciones

Permisos en el manifest

```
<?xml version="1.0" encoding="utf-8"?>
<manifest xmlns:android="http://schemas.android.com/apk/res/android">

    <application
        ...
    </application>

    <uses-permission android:name="android.permission.POST_NOTIFICATIONS" />
    <uses-permission android:name="android.permission.INTERNET" />

</manifest>
```

2. Notificaciones
Alert Dialog

- Un **AlertDialog** es una **ventana emergente** que aparece encima de la pantalla para pedir confirmación al usuario, mostrar un mensaje importante o darle opciones rápidas
- Se usa para cosas como:
  - Confirmar si quiere borrar algo
  - Avisar de un error
  - Pedir una decisión simple (Sí / No)
  - Mostrar un texto importante



2. Notificaciones
Alert Dialog

- onDismissRequest: cuando el usuario toca fuera del dialogo
- confirmButton: composable de botón primario
- dismissButton: composable de botón secundario
- title: titulo de la alerta
- text: texto de la alerta

```
var showDialog by remember { mutableStateOf(false) }

if (showDialog) {
  AlertDialog(
    onDismissRequest = { showDialog = false },
    title = { Text("Eliminar ciclista") },
    text = { Text("¿Seguro que quieres eliminar a ${ciclista.nombre}?") },
    confirmButton = {
      TextButton(
        onClick = {
          showDialog = false
        }
      ) {
        Text("Eliminar")
      }
    },
    dismissButton = {
      TextButton(onClick = { showDialog = false }) {
        Text("Cancelar")
      }
    }
  )
}
```

3. Sensores

Imagen 1
Título asignado a la imagen



Esta foto de Autor desconocido está bajo licencia CC BY-SA-NC

Nota. Adaptado de Título de la imagen, por Autor, Año, Web (con hipervínculo)

3. Sensores
Cámara

- Para manejar la cámara y almacenar las imágenes:
  - 1. Declaramos un FileProvider
  - 2. Configuramos las rutas en res/xml/file_paths.xml
  - 3. En la pantalla creamos el Uri de la foto
  - 4. Lanzamos la cámara con un ActivityResultContracts
  - 5. Dar permisos

3. Sensores

Cámara: FileProvider y permisos

- Un FileProvider sirve para compartir archivos con otras apps usando un content://Uri
- En este caso, se define en el Manifest

```
<application ...>
<provider
   android:name="androidx.core.content.FileProvider"
   android:authorities="${applicationId}.fileprovider"
   android:exported="false"
   android:grantUriPermissions="true">
   <meta-data

android:name="android.support.FILE_PROVIDER_PATHS"
      android:resource="@xml/file_paths" />
</provider>
</application>
```

3. Sensores

Cámara: Configurar las rutas en res/xml/file_paths.xml

- Le decimos al FileProvider que las fotos estarán en: /Android/data/<app>/files/Pictures/
- Es la carpeta que usamos con getExternalFilesDir(Environment.DIRECTORY_PICTURES)
- Creamos un xml con el siguiente contenido:

```
<?xml version="1.0" encoding="utf-8"?>
<paths>
    <!-- Cubre cualquier archivo dentro de getExternalFilesDir(...) -->
    <external-files-path
        name="images"
        path="." />
</paths>
```

3. Sensores

Cámara: Crear el Uri de la foto

- Creamos un archivo temporal ciclista_XXXX.jpg.
- Obtenemos un Uri seguro con FileProvider.getUriForFile(...).
- Ese Uri es el que le pasamos al intent de la cámara.

```
val context = LocalContext.current

fun createImageUri(): Uri {
    val imagesDir =
context.getExternalFilesDir(Environment.DIRECTORY_PICTURES)
    val imageFile = File.createTempFile("ciclista_", ".jpg", imagesDir)
    return FileProvider.getUriForFile(
        context,
        "${context.packageName}.fileprovider",
        imageFile
    )
}
```

3. Sensores

Cámara: Lanzar la cámara

- ActivityResultContracts.TakePicture()
  Es el **contrato** que indica qué tipo de acción se va a realizar: en este caso, **abrir la cámara para hacer una foto y guardarla en un URI**

```
var imageUri by remember {
mutableStateOf<Uri?>(null) }

// Launcher para tomar foto
val takePictureLauncher =
rememberLauncherForActivityResult(
   contract = ActivityResultContracts.TakePicture()
) { success ->
   if (success) {
      // La foto ya está guardada en imageUri
   }
}
```

3. Sensores

Cámara: Ejecutar la captura

- Creamos el Uri del archivo.
- Lanzamos la cámara con ese Uri.
- Si success == true, la foto queda guardada en ese archivo.

```
Button(onClick = {
    val uri = createImageUri()
    imageUri = uri
    takePictureLauncher.launch(uri)
}) {
    Text("Sacar foto")
}
```

3. Sensores
Sensores del móvil

- Los dispositivos de Android disponen de varios sensores incorporados: acelerómetro, giroscopio, luz, etc.
- Para hacer uso de ellos en un app de Android es necesario:
  - SensorManager: gestiona los sensores del móvil y da acceso a ellos
  - SensorEventListener: interface que implementa qué hacer cuando hay cambios en los datos del sensor
  - Exponer los datos en State/Flow

3. Sensores

ViewModel: Ejemplo sensor temperatura

- En el viewModel de la pantalla donde vayamos a utilizar el sensor:
  - Creamos el SensorManager-> sensorManager
  - Creamos el sensor -> sensorManager.getDefaultSensor(Sensor.*TYPE_AMBIENT_TEMPERATURE*)
  - Creamos el listener: sensorManager.registerListener(this, it, SensorManager.*SENSOR_DELAY_NORMAL)*
  - Implementamos la función del cambio en el Sensor-> onSensorChanged(event: SensorEvent)

3. Sensores

ViewModel: Ejemplo sensor temperatura

```
@HiltViewModel
class TemperaturaViewModel @Inject constructor(
    application: Application
) : ViewModel(), SensorEventListener {
    private val sensorManager: SensorManager = application.getSystemService(Context.SENSOR_SERVICE) as SensorManager
    private val tempSensor: Sensor? = sensorManager.getDefaultSensor(Sensor.TYPE_AMBIENT_TEMPERATURE)
    private val _temperature = MutableStateFlow<Float?>(null)
    val temperature: StateFlow<Float?> = _temperature

    init {
        tempSensor?.let {
            sensorManager.registerListener(
                this,
                it,
                SensorManager.SENSOR_DELAY_NORMAL
            )
        }
    }
    override fun onSensorChanged(event: SensorEvent) {
        if (event.sensor.type == Sensor.TYPE_AMBIENT_TEMPERATURE) {
            _temperature.value = event.values[0]   // ºC
        }
    }
...
}
```

3. Sensores

Screen: Ejemplo sensor temperatura

- Se llama al viewModel y se está a la escucha de cambios
  - val viewModel: TemperaturaViewModel = hiltViewModel()
    val temperature by viewModel.temperature.collectAsState()
-

```
@Composable
fun SimulacionScreen() {

   val viewModel: TemperaturaViewModel = hiltViewModel()
   val temperature by
viewModel.temperature.collectAsState()

    ...
    //Contenido de la pantalla
}
```

4. Multimedia

Imagen 1
Título asignado a la imagen



Nota. Adaptado de Título de la imagen, por Autor, Año, Web (con hipervínculo)

4. Multimedia
Audio

- Podemos poner fuentes de audio de nuestra propia app
  - En res/raw insertamos un audio en mp3. Por ejemplo
  - Creamos un estado del tipo MediaPlayer: este objeto manejará el reproductor de video
    - release: libera los recursos
    - pause: pausa la reproducción del sonido
    - Start: activa la reproducción del sonido

4. Multimedia

Ejemplo Audio

```
@Composable
fun AudioPlayerScreen() {
   val context = LocalContext.current

   val mediaPlayer = remember {MediaPlayer.create(context, R.raw.musica) }

   DisposableEffect(Unit) { onDispose {mediaPlayer.release() }
   }
   var isPlaying by remember { mutableStateOf(false) }
   Column(
      modifier = Modifier.fillMaxSize(), horizontalAlignment = Alignment.CenterHorizontally,  verticalArrangement = Arrangement.Center
   ) {
      Text("Reproductor de audio")
      Spacer(Modifier.height(16.dp))
      Button(onClick = {
         if (isPlaying) {
            mediaPlayer.pause()
         } else {
            mediaPlayer.start()
         }
         isPlaying = !isPlaying
      }) {
         Text(if (isPlaying) "Pausa" else "Reproducir")
      }
   }
}
```

4. Multimedia

Video

- Para video podemos utilizar la librería ExoPlayer
  - https://mvnrepository.com/artifact/androidx.media3/media3-exoplayer
  - https://mvnrepository.com/artifact/androidx.media3/media3-ui
- Debemos crear un objeto del tipo ExoPlayer
- Liberarlo en caso de se destruya el composable
- Dentro de AndroidView podemos poner un PlayerView, que nos permite gestionar el reproductor

4. Multimedia

Ejemplo Video

```
@Composable
fun VideoPlayerScreen() {
    val context = LocalContext.current
    val exoPlayer = remember {
        ExoPlayer.Builder(context).build().apply {
            val mediaItem = MediaItem.fromUri("https://www.html5videoplayer.net/videos/toystory.mp4")
            setMediaItem(mediaItem)
            prepare()
            playWhenReady = false
        }
    }
DisposableEffect(Unit) {
        onDispose {
            exoPlayer.release()
        }
    }
AndroidView(
        modifier = Modifier
            .fillMaxWidth()
            .aspectRatio(16 / 9f),
        factory = { ctx ->
            PlayerView(ctx).apply {
                player = exoPlayer
            }
        }
    )
}
```

4. Multimedia
Lector de QR

- Para incorporar un lector de QR a nuestra app es necesario utilizar ML Kit + CameraX
- Debemos añadir las siguientes librerías:
  - androidx.camera:camera-core
  - androidx.camera:camera-camera2
  - androidx.camera:camera-lifecycle
  - androidx.camera:camera-view
  - com.google.mlkit:barcode-scanning

4. Multimedia
Lector de QR

- Una vez incorporadas las librerías debemos crear un composable que muestre la cámara en tiempo real y que escanee los QRs cuando estén delante de ella
- Para ello, es necesario que la app tenga permisos de cámara. En el manifest:
  - <uses-permission android:name="android.permission.CAMERA" />
- El composable recibe un callback onQrScanned que se ejecuta cuando se detecta un QR, pasando el contenido (String).

```
@Composable
fun QRScannerScreen(
    onQrScanned: (String) -> Unit
) {
...
}
```

4. Multimedia
Lector de QR

- Dentro del composable
  - CameraX necesita un LifecycleOwner para arrancar/parar la cámara automáticamente
  - AndroidView permite usar una View tradicional dentro de Compose.

4. Multimedia
Lector de QR

- Dentro del AndroidView
  - Aquí se usa PreviewView (de CameraX) para mostrar la previsualización de cámara.
  - Mostramos el visor de la cámara
  - Obtenemos una instancia del manager de la cámara
  - Cuando el provider está listo se ejecuta
  - Conectamos el preview de la cámara con el previewView de la pantalla

4. Multimedia
Lector de QR

- Dentro del AndroidView
  - Configuramos el escaner del QR con MLKit
  - Configuramos el ImageAnalysis (analizador de frames)
  - Cogemos cada imagen de los frames y los convertimos al formato que pide ML Kit
  - Ejecutamos el scaner de códigos de ML:
    - Si lo lee bien→nos devuelve un string con la información
    - Si no lo lee→lanza un mensaje en log

4. Multimedia
Google Maps

1. Crear un proyecto en Google Cloud Console (https://console.cloud.google.com/)
    - Activar la facturación
    - Habilitar Maps SDK for Android en APIs & Services
2. Crear una API Key y restringirla
    - Crea una API key en “Credentials”.
3. Añadir dependencias
    - com.google.android.gms:play-services-maps
    - com.google.maps.android:maps-compose

4. Multimedia
Google Maps

4. Poner la API key en el AndroidManifest:

```
<meta-data
    android:name="com.google.android.geo.API_KEY"
    android:value="@string/google_maps_key" />
```

En res/values/string.xml:

```
<string name="google_maps_key">TU_API_KEY</string>
```

4. Multimedia
Google Maps

4. Poner la API key en el AndroidManifest:

```
<meta-data
    android:name="com.google.android.geo.API_KEY"
    android:value="@string/google_maps_key" />
```

En res/values/string.xml:

```
<string name="google_maps_key">TU_API_KEY</string>
```

4. Multimedia

Google Maps

```
@Composable
fun MapScreen() {
    val madrid = LatLng(37.99125765, -1.18520193590641)

    val cameraPositionState = rememberCameraPositionState {
        position = CameraPosition.fromLatLngZoom(madrid, 12f)
    }

    GoogleMap(
        modifier = Modifier.fillMaxSize(),
        cameraPositionState = cameraPositionState
    ) {
        Marker(
            state = MarkerState(position = madrid),
            title = "Estoy aquí",
            snippet = "Centro"
        )
    }
}
```

5. Publicar app en Google Play

Imagen 1
Título asignado a la imagen



Nota. Adaptado de Título de la imagen, por Autor, Año, Web (con hipervínculo)

5. Publicar app en Google Play

1. Crear cuenta de Google Play Developer
   - https://play.google.com/console
   - Pago de 25 dólares
2. Preparar la app para producción
   - Cambiar versión y nombre del paquete
   - Quitar las cosas de desarrollo (logs, datos pruebas, permisos, etc.)
   - Iconos y nombre
3. Generar el archivo de publicación (Android App Bundle (AAB)
   - Generar un keystore (guardar porque si se pierde no se puede actualizar)

5. Publicar app en Google Play

4. Crear la app en Google Play Console
    - Elige idioma, nombre, app o juego y si es de pago o gratis (irreversible)
5. Subir el AAB
6. Ficha de la app
7. Define las políticas y formularios obligatorios
8. Envío a revisión (1-7 días)
9. Aceptación y publicación

REFERENCIAS

BIBLIOGRÁFICAS

OpenCompose: https://www.jetpackcompose.pro/

Exoplayer: https://developer.android.com/media/media3/exoplayer/hello-world?hl=es-419

Descripción general CameraX: https://developer.android.com/media/camera/camerax?hl=es-419

Analizador de ML Kit: https://developer.android.com/media/camera/camerax/mlkitanalyzer?hl=es-419

Daniel Valero Carreras

dvalero@ucam.edu

UCAM Universidad Católica de Murcia



© UCAM